

Gate-Based Quantum Optimization for Discrete Process Design: A Repair-Aware Benchmark

Yirang Park^a and David E. Bernal Neira^a

^a Davidson School of Chemical Engineering, Purdue University, West Lafayette, IN 47907, USA

ABSTRACT

Discrete choices of routes, units, and operating modes form the first combinatorial barrier in process design. We audit what is required to turn gate-based quantum samples into interpretable design candidates on a fully enumerable drug-substance manufacturing instance with 19 binary flow variables. The penalty QUBO reformulation adds 17 auxiliary variables; a sampled 36-bit state becomes a design candidate only after projection onto the flow variables and exact auxiliary repair. The 19-variable surrogate and transferred QAOA angles rely on exhaustive enumeration with repair, which is feasible only for this small instance. The workflow therefore tests how to interpret quantum samples; it does not provide a scalable optimization method. After repair-aware preprocessing, energy-targeted transferred-angle QAOA concentrates probability on the optimal flowsheet (664 of 262144 ideal reads) but does not outperform a simple hill climber, an exact solver, or exhaustive enumeration. A small IBM hardware run verifies the pipeline without adding advantage. These results are a workflow-feasibility and interpretation study, not a claim of quantum speedup.

Keywords: quantum computing, process design, discrete optimization, QUBO reformulation, pharmaceutical manufacturing

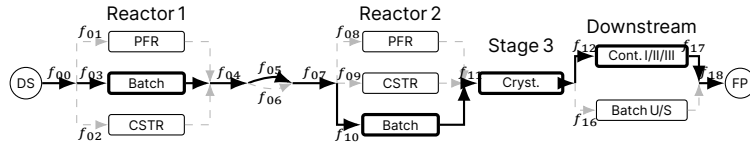
INTRODUCTION

Discrete choices are often the first barrier in process design. Before a flowsheet can be simulated or optimized over continuous variables, the designer must commit to a set of mutually exclusive options: which synthesis route to use, which unit operations to include, and which operating modes to run [1]. In pharmaceutical manufacturing, these choices are particularly consequential because route and equipment selection shape cost, yield, and regulatory burden, and because digital design and optimization tools for this sector are comparatively recent [2, 3]. The resulting models are combinatorial, and even modestly sized superstructures generate large discrete search spaces.

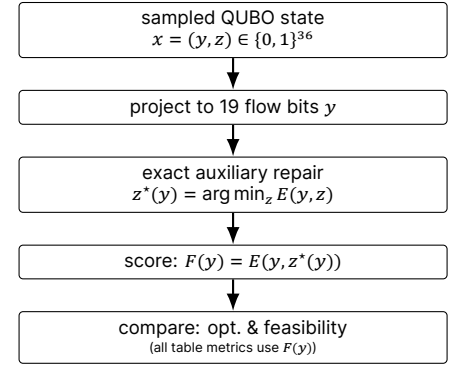
Gate-based quantum optimization is one of the emerging tools proposed for this setting. A gate-based quantum computer operates on quantum bits (qubits) by applying

sequences of parameterized gate operations that form a quantum circuit; measuring the circuit returns a binary string, and the circuit parameters, called angles, are tuned so that low-energy strings appear with high probability. The quantum approximate optimization algorithm (QAOA) [4] and the variational quantum eigensolver (VQE) [5, 6] follow this variational paradigm: QAOA uses p alternating cost and mixer layers, and VQE uses a parameterized ansatz circuit (here, a hardware-efficient ansatz); in both cases, a classical optimizer adjusts the angles. Current hardware is noisy and of intermediate scale [7], and benchmarking surveys stress that a practical quantum advantage for optimization has not yet been established [8]; the relevant question is what is required to use these methods today and how their output compares with classical baselines.

Within process systems engineering, quantum comput-



(a) DS-MFG manufacturing superstructure (19 binary arcs)



(b) Repair-aware scoring pipeline

Figure 1. Overview of the DS-MFG case study. **(a)** Manufacturing superstructure: thick arrows trace the optimal flowsheet (Batch Rxn 1 → Route A (f_{05}) → Batch Rxn 2 → Cryst. → Cont. I (f_{13})); dashed arrows mark the non-optimal alternatives; the backbone arcs $f_{00}, f_{04}, f_{07}, f_{11}, f_{18}$ carry every feasible flow. Route B (f_{06}) is activated only when a continuous Reactor 1 feeds a batch Reactor 2; Cont. I/II/III share entry arc f_{12} and exit arc f_{17} (optimal uses f_{13}). **(b)** Every sampled 36-bit QUBO string is projected to the 19 flow variables, repaired exactly via the 13-component auxiliary decomposition (GitHub repository), and scored against the certified optimum and the 36-member feasible-design pool; hit rates appear in Table 1.

ing has been reviewed as an emerging capability [9], and hybrid quantum-classical strategies have been proposed for discrete optimization in process and energy systems [10]. Discrete-design instances have been mapped to the QUBO form [11] and solved with Ising-based annealing hardware [10, 12]. These works establish that the mapping is possible; few examine what happens to the samples after the reformulation, and the gate-based path is less explored than annealing. Casting a constrained problem as a QUBO introduces auxiliary variables [13] that are bookkeeping rather than design decisions, so a raw quantum sample is not directly interpretable as a flowsheet. The question is: what does it take to turn gate-based quantum samples into process-design decisions, and how does this compare with classical methods?

We answer this through an end-to-end audit on a single drug-substance manufacturing (DS-MFG) instance, chosen to be small enough to enumerate exactly yet large enough to expose the reformulation and interpretation issues that arise at scale. Our contribution is a repair-aware audit of the full path from a constrained process-design model to usable gate-based quantum samples. We find: (i) raw QUBO samples require projection and exact auxiliary repair before they are interpretable as design candidates; (ii) a quadratic surrogate over the design variables fits the repaired landscape globally but fails as an optimizer; (iii) transferred-angle QAOA concentrates probability on good flowsheets but does not beat classical baselines; and (iv) a hardware run confirms pipeline feasibility without adding advantage. Because the 19-

variable surrogate and transferred angles are built using exhaustive repair enumeration, feasible only for this small instance, this workflow serves as a diagnostic benchmark for sample interpretation, not a scalable optimization shortcut.

For the process systems engineering community, this study is a concrete, reproducible measurement of the engineering burden between a discrete-design model and a usable quantum sample; the conclusions demonstrate feasibility and interpretation, not a runtime advantage over classical computation.

METHODS

The Discrete Design Instance

The DS-MFG instance is the discrete subproblem of a drug-substance manufacturing case study introduced by Park and Bernal Neira [12], where a mixed-integer nonlinear design problem is decomposed so that the discrete part is handled by combinatorial solvers and the nonlinear part by simulators. We keep the same discrete instance but ask what is required to run gate-based optimizers. The instance is a binary integer program for capital-expenditure selection in the manufacturing superstructure: the 19 binary variables encode candidate material-flow arcs and unit-operation selections, and the objective function is a normalized capital-cost score (lower values indicate lower capital expenditure; scaled as in [12]). The

discrete problem has the form

$$\begin{aligned}
& \min_f \sum_{i \in F} c_i f_i \\
& \text{s.t.} \quad \sum_{i \in \text{In}(n)} f_i = \sum_{j \in \text{Out}(n)} f_j \quad \forall n \in N, \\
& \quad f_{\text{source}} = 1, \quad f_{\text{sink}} = 1, \\
& \quad g(f) \leq 0, \\
& \quad f_i \in \{0, 1\} \quad \forall i \in F.
\end{aligned} \tag{1}$$

Eq. (1) lists the relevant constraint classes: flow balance at each node $n \in N$, fixed source and sinks, and additional constraints $g(f) \leq 0$ that enforce a valid flowsheet.

Solved with `Gurobi` [14], the model (19 binary variables, 20 constraints) returns an optimal capital-cost score of 11.7095; with feasible solutions enumeration enabled (`PoolSearchMode=2`), all 36 feasible flows are retained in a single deterministic solve (0.073 s). We treat 11.7095 as the certified global optimum and use it as the reference throughout.

From Integer Program to QUBO and Back

To run QAOA and VQE, the integer program must be expressed as a QUBO, in which all constraints are folded into a single quadratic objective over binary variables [11, 13]. Inequality constraints require auxiliary (slack) variables; the penalty weights and auxiliary count are not unique, and both affect how faithfully the QUBO represents the original problem [13]. For this instance, the penalty reformulation introduces 17 auxiliary variables, so the QUBO acts on 36 binary variables. For a bitstring $x \in \{0, 1\}^{36}$, the encoded objective is

$$E_{\text{QUBO}}(x) = \ell^T x + x^T Q x + b, \tag{2}$$

with linear coefficients ℓ , quadratic coefficient matrix Q , and constant offset b .

The 36 variables are separated into the original flow variables and the auxiliaries, $x = (y, z)$, $y \in \{0, 1\}^{19}$, $z \in \{0, 1\}^{17}$. Only y encodes a design decision; z is bookkeeping. A sampled 36-bit string does not map directly to a flowsheet. We make it interpretable by projecting onto y and repairing the auxiliaries, choosing the assignment that minimizes the QUBO energy for that flow,

$$F(y) = \min_{z \in \{0, 1\}^{17}} E_{\text{QUBO}}(y, z). \tag{3}$$

The repair is exact and cheap here because, once y is fixed, the auxiliary-auxiliary coupling graph splits into 13 independent connected components: 9 singletons (one auxiliary bit each) and 4 pairs (two auxiliary bits

each). The singletons correspond to the at-most-one inequality constraints for each decision group and to the downstream flow-balance conditions; the pairs arise from product-linearization inequalities for the Route B activation rule ($f_{06} = 1$ iff Reactor 1 is continuous and Reactor 2 is batch). Each component is solved by exhaustive enumeration over its 2 or 4 possible assignments; the $9 \times 2 + 4 \times 4 = 34$ local assignment evaluations replace enumeration of $2^{17} = 131,072$ auxiliary states. The full component table with auxiliary indices, flow neighbors, and constraint roles appears in the Supplement. This decomposition is instance-specific; for a general QUBO, auxiliary repair can itself be NP-hard, so the present repair step is an exact audit tool rather than a generally cheap postprocessor.

The repaired minimizer over all 2^{19} flows recovers the optimum, and the repaired QUBO scores match the integer-program objective to within 3.1×10^{-13} , confirming they encode the same objective. Exact enumeration finds exactly 36 feasible flows, as does the solution pool option of `Gurobi`. The 36 feasible flows occupy repaired ranks 1–36; the 14 infeasible flows in the top 50 follow them.

The repaired objective $F(y)$ is not a quadratic function of y , but QAOA and VQE require a QUBO input. Rather than using the full 36-variable QUBO (which mixes design variables with auxiliaries and requires 36-qubit circuits), we construct a reduced, 19-variable QUBO directly over the design space y by fitting a quadratic surrogate to the repaired landscape:

$$\hat{F}(y) = \hat{b} + \hat{\ell}^T y + y^T \hat{Q} y \tag{4}$$

by least squares over all 2^{19} repaired flows. The surrogate fits the landscape almost perfectly in aggregate ($R^2 \approx 0.999$), but this global accuracy is misleading for optimization: the surrogate's own minimizer repairs to 14.5505 rather than the true 11.7095, the true optimum ranks only 33rd under the surrogate, and among the 50 lowest-cost repaired flows the surrogate's absolute errors reach 29.4. The surrogate is not used to certify solution quality; it serves only as a compact 19-qubit sampling Hamiltonian, and every reported metric is evaluated after exact repair under F .

Quantum, Classical, and Hardware Workflows

We evaluate several workflows over this pipeline. The local QAOA and VQE workflows run through `QiskitOpt.jl`, a Julia wrapper for `Qiskit` [15], with QUBO models constructed via the `QUBO.jl` ecosystem [13]. Increasing p makes QAOA more expressive and can be viewed as a finer discretization of an adiabatic path [16], but deeper circuits are more sensitive to noise on current NISQ devices [7]. The native `Qiskit` searches use

COBYLA; the offline JuliQAOA search uses basin hopping with BFGS local optimization [6, 17]. Direct full-QUBO QAOA and VQE operate on the 36-variable formulation; reduced-surrogate QAOA and VQE operate on the 19-variable surrogate and are scored by exact repair. The best-performing quantum workflow adds an offline parameter-search step: QAOA parameters are optimized on the surrogate with JuliQAOA [17] by minimizing the expected surrogate energy $\mathbb{E}_{y \sim p_\theta}[\tilde{F}(y)]$, then transferred to a Qiskit circuit for sampling (*energy-targeted*: angles are optimized offline against the surrogate expected energy; this offline cost is excluded from the sampling times in Table 1). The pre-optimized parameters with the highest local solution concentration are also submitted to the hardware for a feasibility test. Surrogate VQE uses an `EfficientSU2` ansatz optimized with COBYLA.

As classical baselines, we sample the repaired 19-flow objective by uniform random sampling and by single-bit-flip hill climbing with random restarts. As a hardware feasibility test, we run the highest-concentration fixed-parameter QAOA circuit on the IBM `ibm_fez` quantum computer. An independent state-vector calculation predicts the central QAOA optimum probability at 2.63×10^{-3} , consistent with the 2.53×10^{-3} sampled rate.

For each sampled state $x = (y, z)$ we record the raw QUBO energy $E_{\text{QUBO}}(y, z)$, the repaired objective $F(y)$, and the auxiliary gap. A *reference-optimum hit* means the repaired flow equals the certified optimal flowsheet (the unique minimum-cost feasible design). A *feasible-design hit* means the repaired flow matches any of the 36 feasible flows. We report both hit rates with two-sided Wilson 95% intervals; zero-hit rows show their upper endpoint. The empirical $\text{TTS}_{99} = \tau[\log(0.01)/\log(1 - \hat{p})]$ uses the recorded call time per sample τ ; zero-hit rows report ∞ . These values compare sampling costs and exclude pre-processing. Heuristic evaluations are correlated, and VQE performance varies with the seed, so the intervals and TTS values do not guarantee success across repeated runs.

RESULTS

In Table 1, transferred-angle QAOA gives the highest stochastic feasible-design rate, but hill climbing and Gurobi remain faster on the recorded cost metrics.

Repair Changes the Meaning of a Sample

Sampling the 36-variable QUBO directly does not yield an interpretable design result. A $p = 2$ full-QUBO QAOA found no optimum in 512 samples; a 32768-read run returned it only 4 times after repair, and never as a fully optimal 36-bit string without projection. Without projection

and exact repair, a raw QUBO sample cannot be read as a flowsheet; Figure 1(b) summarizes the scoring pipeline, and the repair decomposition is detailed in the Supplement.

Reduction and Parameter Transfer Concentrate QAOA Samples

Once the pipeline is in place, reduction to the surrogate and transferred-parameter search are what make QAOA samples concentrate on feasible, optimal flowsheets. The energy-targeted $p = 5$ transferred workflow sampled the certified optimum 664 times in 262144 ideal reads and matched a feasible flowsheet 32502 times (12.4%, CI [12.3, 12.5]%), the highest feasible-design rate of any workflow; see Table 1 for TTS99 values. The surrogate's near-perfect global R^2 did not imply optimization fidelity: its own minimizer repaired to 14.5505, and the true optimum ranked only 33rd under the surrogate.

VQE and Classical Baselines Bound the Claim

VQE benefits from the same reduction but produces a flatter distribution. Full-QUBO VQE found no optimum in a five-seed sweep. Table 1 reports all 20 initial reduced-surrogate runs: seeds 74001–74020, 32768 reads each, with 3 optimum and 61 feasible-design hits in 655360 reads. Only seeds 74001 and 74007 found the optimum (1 and 2 hits); the other 18 found none. Hill climbing found the optimum 19 times in 262144 evaluations; despite a lower hit rate than QAOA, its much smaller per-sample cost gives a better TTS99-opt (0.23 s vs. 0.37 s for QAOA). Random sampling never found the optimum. Thus, QAOA gives the highest stochastic feasible-design rate but does not beat classical baselines on time-to-solution.

Hardware Execution Is an Implementation Feasibility Check

To check that the implementation runs on hardware, we submitted the highest-concentration fixed-angle QAOA circuit to the `ibm_fez` quantum computer across three repeats and three random transpile seeds (92001–92003), with 4096 measurement shots per repeat, for 36864 total hardware reads (9 jobs, 73.5 s elapsed including cloud queue). The run produced 4 feasible-design reads and no reference-optimum read; the best hardware sample was the second-cheapest feasible flowsheet (objective 11.8105). Using this recorded time gives a TTS99-feas of 84.6 s, versus 0.006 s for hill climbing and 0.073 s for the deterministic Gurobi solve. Queue time was not recorded separately, so we cannot determine how much it contributed. The pilot used angles optimized to favor the known top 10 flows; the energy-targeted QAOA row in Table 1 uses different angles. With zero optimum hits, the

Table 1: Primary outcomes after exact auxiliary repair. Opt rate: certified-optimum hit rate; feas rate: any of the 36 feasible designs. Wilson 95% intervals [lo, hi] use the same units as the rate; zero-hit rows show the upper endpoint. Call time covers the recorded solver call or hardware run. It excludes prior preprocessing and any scoring performed after the call. TTS99 uses these call times and does not measure the full workflow or guarantee success across seeds (Methods). Gurobi reports a deterministic solve time.

Workflow	Call time (s)	Opt rate [95% CI]	Feas rate [95% CI]	TTS99-opt (s)	TTS99-feas (s)
Gurobi solution pool	0.073	Deterministic optimum	36/36 (exhaustive)	0.073	0.073
Uniform random sampling	2.08	$< 1.5 \times 10^{-5}$	$8.0 [5.2, 12.2] \times 10^{-5}$	∞	0.46
Hill-climb restarts	0.93	$7.2 [4.6, 11.3] \times 10^{-5}$	$2.75 [2.56, 2.96] \times 10^{-3}$	0.23	0.006
Energy-targeted QAOA, $p=5$	53.9	$2.53 [2.35, 2.73] \times 10^{-3}$	$0.124 [0.123, 0.125]$	0.37	0.007
Reduced-surrogate VQE, all 20 seeds	45.4	$4.6 [1.6, 13.5] \times 10^{-6}$	$9.3 [7.2, 12.0] \times 10^{-5}$	69.7	3.43
IBM <code>ibm_fez</code> pilot	73.5	$< 1.1 \times 10^{-4}$	$1.1 [0.42, 2.8] \times 10^{-4}$	∞	84.6

Key counts: QAOA $p=5$: 664 opt. hits and 32502 feas. hits in 262144. IBM `ibm_fez`: 0 opt. hits and 4 feas. hits in 36864.

Wilson 95% upper bound on the hardware optimum-hit rate is about 1.04×10^{-4} .

DISCUSSION AND LIMITATIONS

This study asks what is needed to turn QAOA and VQE samples into meaningful process-design decisions. The steps are to separate design and auxiliary variables, repair the auxiliaries exactly, fit a quadratic surrogate, optimize the angles offline, and score samples in the original flow space. Even when the samples concentrate on good flow-sheets, classical methods remain faster for this instance.

Robustness and Additional Classical Baselines

We tested how the results change with penalty weights, surrogate training data, QAOA depth, optimizer settings, and random seeds. The repository's `ds_mfg_revision_audit/` directory records the settings, seeds, results, and timings for each run. For the penalty test, write the QUBO as capital cost $c(y)$ plus penalty $P(y, z)$ and scale only the penalty: $F_\alpha(y) = c(y) + \alpha \min_z P(y, z)$. Among the tested values $\alpha = 0.01, 0.05, 0.1, 0.25, 0.5, 1, 2, 4$, exact enumeration gives the same certified optimum for $\alpha \geq 0.25$; the three smaller weights give infeasible minima. All feasible designs rank ahead of every infeasible design for tested $\alpha \geq 0.5$. At the submitted weight ($\alpha = 1$), the lowest infeasible score exceeds the highest feasible score by 31.3717. These results apply to this instance with all penalty terms scaled together.

We also fitted quadratic surrogates to uniform random subsets of 1024, 8192, and 65536 flows, using unweighted least squares and three seeds per size. Their global R^2 values range from 0.99884 to 0.99908, but their minimizers repair to 14.5505–18.1755. None finds the true optimum, despite fitting the overall landscape closely. For

the fit using all flows, a degree-two Walsh projection reproduces the unweighted least-squares solution.

New energy-targeted QAOA runs use $p = 1$ –5, three random seeds, and one basin-hopping iteration. The optimum probability rises from 1.42×10^{-4} at $p = 1$ to $(2.63$ – $2.81) \times 10^{-3}$ at $p = 5$ across seeds. For seed 91001, increasing basin-hopping iterations from one to five leaves the $p = 5$ optimum probability at 2.63×10^{-3} , while the time to optimize the entire $p = 1$ –5 sequence increases from 103.6 to 300.5 s. The effect of increasing the optimizer budget was tested at this one seed.

The VQE follow-up was exploratory. Seed 74018 had the highest top-50 count in the initial 20-seed study (23 hits), but no optimum hit. Its later 20-hit result therefore does not represent performance across initializations. At 524288 reads per seed, the selected follow-ups give 5, 3, and 20 optimum hits for seeds 74001, 74007, and 74018; later reruns to retain circuit metadata give 3, 1, and 0. The archive lists all 43 cached VQE runs, with different settings and later reruns reported separately.

We compared simulated annealing [18], uniform sampling, and hill climbing using five common seeds and 262144 evaluations per run, each starting with an empty repair cache. Annealing uses geometric cooling from 100 to 0.01 and a random restart every 4096 evaluations. Both annealing and hill climbing find the optimum in all five runs; uniform sampling does so in three. These counts describe how often each search found the optimum within its evaluation budget.

Computational Cost and Scope

Total runtime includes model reformulation, exhaustive enumeration and repair, surrogate fitting, offline optimization, circuit construction and compilation, sampling, hardware queue and execution time, result retrieval, and

final repair and scoring. The original runs did not retain preprocessing or offline-search times, and the 73.5 s hardware record does not separate compilation, queueing, and execution. We therefore cannot reconstruct the original total runtime; the missing stages still contribute to the cost. Separate CPU reruns took 2.3 s to enumerate and repair all flows, 0.005 s for the equivalent fit using all flows, and 0.054 s to build the logical circuit. Compilation on the offline FakeFez model took 0.076 s (median of three transpiler seeds). The archive reports what each timer includes, startup and warmup costs, and the 300.5 s of offline search. These new measurements are kept separate from the original sampling times.

Exact enumeration finds the optimum before quantum sampling begins, so these experiments test how to interpret samples after repair. Repair is cheap here because the auxiliary graph has 13 small components; in general, it can be NP-hard. Enumeration and dense statevector simulation each grow as 2^n , while repair by component enumeration requires $\sum_c 2^{|c|}$ local assignments. Larger problems would need approximations whose effects on feasibility and optimality are checked. Tests on this one 19-flow instance cannot establish scalability or a total runtime advantage for larger process designs [8].

CONCLUSION

For this drug-substance manufacturing case study, QAOA and VQE samples become interpretable as process-design decisions only after projection to the flow variables and exact auxiliary repair. Transferred-angle QAOA concentrates probability on good flowsheets but does not beat classical baselines on any time-to-solution metric (Table 1). The central lesson is that quantum-sample quality must be evaluated after mapping back to the original process decisions; raw QUBO samples can overstate both feasibility and optimization performance, and honest benchmarks must account for the full preprocessing cost.

ACKNOWLEDGMENTS

This material is based upon work supported by the Center for Quantum Technologies under the Industry-University Cooperative Research Center Program at the US National Science Foundation under Grant No. 2224960. Scripts, data files, and circuit records are archived at <https://github.com/SECQUOIA/pse-quantum-discrete-optimization>.

DECLARATION OF USE OF AI

The authors used generative AI tools to assist with initial code drafting, data-analysis scripting, and draft vi-

sualization design. All code, analyses, visualizations, and manuscript text were reviewed, corrected, and revised by the authors, who take full responsibility for the accuracy and integrity of all results.

AUTHOR IDENTIFIERS

Author ORCIDs:

Park: 0009-0008-6629-3308

Bernal Neira: 0000-0002-8308-5016

REFERENCES

- [1] L. Mencarelli, Q. Chen, A. Pagot, and I. E. Grossmann. "A review on superstructure optimization approaches in process system engineering". In: *Computers and Chemical Engineering* 136 (2020), p. 106808. DOI: [10.1016/j.compchemeng.2020.106808](https://doi.org/10.1016/j.compchemeng.2020.106808).
- [2] D. Casas-Orozco, D. Laky, V. Wang, M. Abdi, X. Feng, E. Wood, C. Laird, G. V. Reklaitis, and Z. K. Nagy. "PharmaPy: An object-oriented tool for the development of hybrid pharmaceutical flowsheets". In: *Computers & Chemical Engineering* 153 (2021), p. 107408. DOI: [10.1016/j.compchemeng.2021.107408](https://doi.org/10.1016/j.compchemeng.2021.107408).
- [3] D. Laky, D. Casas-Orozco, C. D. Laird, G. V. Reklaitis, and Z. K. Nagy. "Simulation-Optimization Framework for the Digital Design of Pharmaceutical Processes Using Pyomo and PharmaPy". In: *Industrial & Engineering Chemistry Research* 61.43 (2022), pp. 16128–16140. DOI: [10.1021/acs.iecr.2c01636](https://doi.org/10.1021/acs.iecr.2c01636).
- [4] E. Farhi, J. Goldstone, and S. Gutmann. *A Quantum Approximate Optimization Algorithm*. 2014. arXiv: [1411.4028](https://arxiv.org/abs/1411.4028) [quant-ph]. URL: <https://arxiv.org/abs/1411.4028>.
- [5] A. Peruzzo, J. McClean, P. Shadbolt, M.-H. Yung, X.-Q. Zhou, P. J. Love, A. Aspuru-Guzik, and J. L. O'Brien. "A variational eigenvalue solver on a photonic quantum processor". In: *Nature Communications* 5 (2014), p. 4213. DOI: [10.1038/ncomms5213](https://doi.org/10.1038/ncomms5213). arXiv: [1304.3061](https://arxiv.org/abs/1304.3061) [quant-ph].
- [6] M. Cerezo, A. Arrasmith, R. Babbush, S. C. Benjamin, S. Endo, K. Fujii, J. R. McClean, K. Mitarai, X. Yuan, L. Cincio, and P. J. Coles. "Variational quantum algorithms". In: *Nature Reviews Physics* 3 (2021), pp. 625–644. DOI: [10.1038/s42254-021-00348-9](https://doi.org/10.1038/s42254-021-00348-9). arXiv: [2012.09265](https://arxiv.org/abs/2012.09265) [quant-ph].
- [7] J. Preskill. "Quantum Computing in the NISQ era and beyond". In: *Quantum* 2 (2018), p. 79. DOI: [10.22331/q-2018-08-06-79](https://doi.org/10.22331/q-2018-08-06-79).

- [8] A. Abbas, A. Ambainis, B. Augustino, et al. “Challenges and opportunities in quantum optimization”. In: *Nature Reviews Physics* 6.12 (2024), pp. 718–735. DOI: [10.1038/s42254-024-00770-9](https://doi.org/10.1038/s42254-024-00770-9).
- [9] D. E. Bernal, A. Ajagekar, S. M. Harwood, S. T. Stober, D. Trenev, and F. You. “Perspectives of quantum computing for chemical engineering”. In: *AIChE Journal* 68.6 (2022), e17651. DOI: [10.1002/aic.17651](https://doi.org/10.1002/aic.17651).
- [10] A. Ajagekar and F. You. “Quantum computing for energy systems optimization: Challenges and opportunities”. In: *Energy* 179 (2019), pp. 76–89. DOI: [10.1016/j.energy.2019.04.186](https://doi.org/10.1016/j.energy.2019.04.186).
- [11] F. Glover, G. Kochenberger, and Y. Du. *A Tutorial on Formulating and Using QUBO Models*. 2019. arXiv: [1811.11538](https://arxiv.org/abs/1811.11538) [cs.DS]. URL: <https://arxiv.org/abs/1811.11538>.
- [12] Y. Park and D. E. Bernal Neira. *Leveraging Classical and Quantum Computing for Process Systems Engineering Applications: Decomposition Algorithm with Ising Solvers for Efficient Discrete Landscape Exploration*. 2026. arXiv: [2603.19520](https://arxiv.org/abs/2603.19520) [math.OC]. URL: <https://arxiv.org/abs/2603.19520>.
- [13] P. M. Xavier, P. Ripper, T. Andrade, J. D. Garcia, N. Maculan, and D. E. Bernal Neira. *QUBO.jl: A Julia ecosystem for quadratic unconstrained binary optimization*. 2023. DOI: [10.48550/arXiv.2307.02577](https://doi.org/10.48550/arXiv.2307.02577). arXiv: [2307.02577](https://arxiv.org/abs/2307.02577) [math.OC]. URL: <https://arxiv.org/abs/2307.02577>.
- [14] Gurobi Optimization, LLC. *Gurobi Optimizer Reference Manual*. 2026. URL: <https://www.gurobi.com>.
- [15] A. Javadi-Abhari, M. Treinish, K. Krsulich, C. J. Wood, J. Lishman, J. Gacon, S. Martiel, P. D. Nation, L. S. Bishop, A. W. Cross, B. R. Johnson, and J. M. Gambetta. *Quantum computing with Qiskit*. 2024. DOI: [10.48550/arXiv.2405.08810](https://doi.org/10.48550/arXiv.2405.08810). arXiv: [2405.08810](https://arxiv.org/abs/2405.08810) [quant-ph].
- [16] E. Farhi, J. Goldstone, S. Gutmann, and M. Sipser. *Quantum Computation by Adiabatic Evolution*. 2000. arXiv: [quant-ph/0001106](https://arxiv.org/abs/quant-ph/0001106). URL: <https://arxiv.org/abs/quant-ph/0001106>.
- [17] J. Golden, A. Bärtschi, D. O'Malley, E. Pelofske, and S. Eidenbenz. “JuliQAOA: Fast, Flexible QAOA Simulation”. In: *SC-W '23: Proceedings of the Workshops of The International Conference on High Performance Computing, Network, Storage, and Analysis*. Association for Computing Machinery, 2023, pp. 1454–1459. DOI: [10.1145/3624062.3624220](https://doi.org/10.1145/3624062.3624220). arXiv: [2312.06451](https://arxiv.org/abs/2312.06451) [quant-ph].
- [18] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi. “Optimization by simulated annealing”. In: *Science* 220 (4598 1983), pp. 671–680. ISSN: 00368075. DOI: [10.1126/SCIENCE.220.4598.671](https://doi.org/10.1126/SCIENCE.220.4598.671). URL: <https://www.science.org>.

© 2027 by the authors. Licensed to PSEcommunity.org and PSE Press. This is an open access article under the creative commons CC-BY-SA licensing terms. Credit must be given to creator and adaptations must be shared under the same terms. See <https://creativecommons.org/licenses/by-sa/4.0/>.

